Assignment #5

Kabir Kanha Arora
CSD 302 - Design and Analysis of Algorithms
Shiv Nadar University

April 20, 2020

Question 1

Given two character strings x[1..n] and y[1..m], the edit distance is the cost of transforming the string x to y using a minimum number of operations from the set {copy, replace, insert, delete}. Design an efficient algorithm to find the minimum edit distance between two given strings.
What if there are specific costs associated with each of the operation and you want to minimize the total cost?
This has direct application to DNA sequencing problem, i.e. how close they are to each other.

This is a Dynamic Programming question and demands a DP table. The table is set up in a way the the string x is split along the rows and the string y is split along the columns, but leaving the entry for the first row and first column blank.
Before explaining the algorithm, let me illustrate with an example where $x = "SEN"$ and $y = "SANDEEP"$. The DP table would then like like this initially:

	Φ	S	A	N	D	E	E	P
Φ	0	1	2	3	4	5	6	7
S	1							
E	2							
N	3							

Each (i, j) in the table reflects the cost of converting the string $x[0...i]$ to the string $y[0...j]$. Hence, the bases cases are reflected in the Φ column and row where they represent i deletions or j insertions only. The numbers in the table reflect the number of operations required to transform the string. According to me, we don't need to use *copy* anywhere and I am only considering *replace*, *insert* and *delete*. *Copy* would have made sense in case we were storing the new string in another variable, but here we are only 'transforming' x into y and copy does not seem to serve any purpose.

Now, we get down to actually filling the table. Clearly, the required edit distance between the two strings is stored in the bottom right cell of the table. We start filling the table from the top left in a sequential fashion. For each cell (i, j), we compare the corresponding

characters in x and y. If they are equal, the edit distance is the edit distance is equal to the the edit distance for the strings preceding these characters i.e., it is equal to the $(i-1, j-1)^{th}$ entry.

If these two characters are not equal, we can do three things:

1. We can look at the $(i-1, j-1)^{th}$ and add 1 to it. This is equivalent to **replacing** the last character in $x[0...i]$ to make it equal to $y[0...j]$.

2. Another option is to look at the $(i-1, j)^{th}$ and add 1 to it. This represents **deleting** the last character from $x[0...i]$.

3. The final option is to look at $(i, j-1)^{th}$ and add 1 to it, which effectively means that we **copy** the last character from $y[0...j]$.

We choose the option that gives us the minimum edit distance. By this algorithm, the table looks like this at the end:

	Φ	S	A	N	D	E	E	P
Φ	0	1	2	3	4	5	6	7
S	1	0	1	2	3	4	5	6
E	2	1	1	2	3	3	4	5
N	3	2	2	1	2	3	4	**5**

Here, we see, that the edit distance is 5, which does seem to be the correct answer. The time and space complexities of this algorithm are both $\mathcal{O}(m * n)$

Now, let us consider what happens if there are specific costs associated with each of the functions. Let the cost for *copy* be c, the cost for *delete* be d and the cost for *replace* be r. Now the only modification we need to make to our algorithm is that in the three possible options when the characters do not match, we add r, d and c instead of 1 and then choose the operation with the minimum cost.

Question 2

There are n destinations D_i, $1 \leq i \leq n$ with demands d_i.
There are two warehouses W1 and W2 that have inventory $r1$ and $r2$ respectively such that

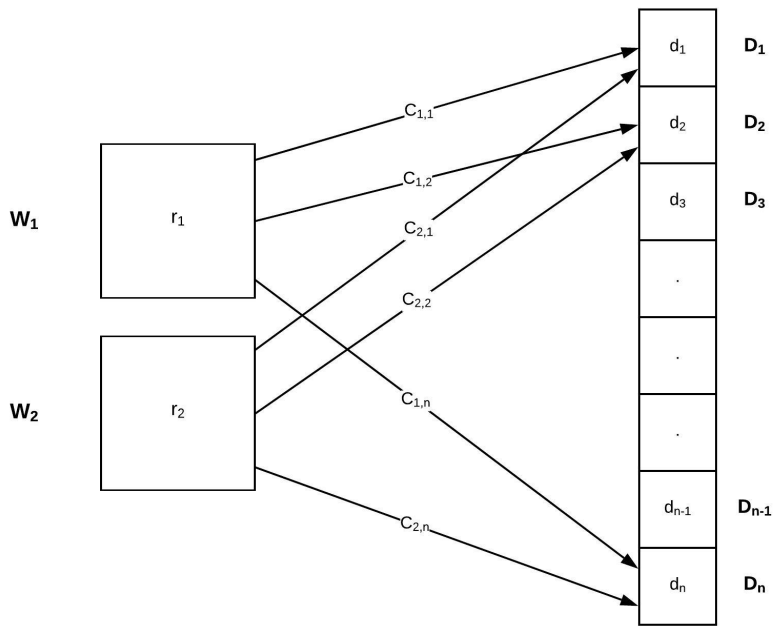
$$\sum_{i=1}^{n} d_i = r1 + r2$$

The cost of transporting $x_{i,j}$ units from W_i to D_j is $c_{i,j}(x_{i,j})$. We must ensure that $x_{1,j} + x_{2,j} = d_j$ in a way so as to minimize

$$\sum_{i=1}^{\infty} \sum_{i=1}^{\infty} c_{i,j}(x_{i,j})$$

Hint: Let $g_i(x)$ be the cost incurred when W1 has an inventory of x and supplies are sent to $D_1...D_i$ in an optimal manner - the inventory at W2 is $\sum_{j=1}^{i} d_j - x$

This problem can be visualised with the help of the following diagram:



Here, we need to minimise the cost. So the most simple solution that comes to my mind is that we can maintain an map of n elements, where the ith value corresponds to $c_{1,i} - c_{2,i}$, and the keys are the destination indices. Then we simply sort the map in decreasing order of values and traverse through the values, and fulfilling the demands of the destinations with W_2, since choosing W_1 would give decreasingly bad results for these values. Then once the inventory of W_1 has been exhausted, we supply the remainder of the demands for the destinations from W_2.
What this means is that we try to completely fulfil demands of as many destinations in this sorted list with W_2 as possible, before moving on to W_1.

Question 3

Graph theory
(i) Show that in any graph there are at least two vertices of the same degree.
(ii) Given a degree sequence $d_1, d_2...d_n$ such that $\sum_{i=1}^{n} d_i = 2n - 2$, construct a tree whose vertices have the above degrees.
(iii) Show that in a complete graph of six vertices where edges are colored red or blue, there is either a red or a blue triangle.

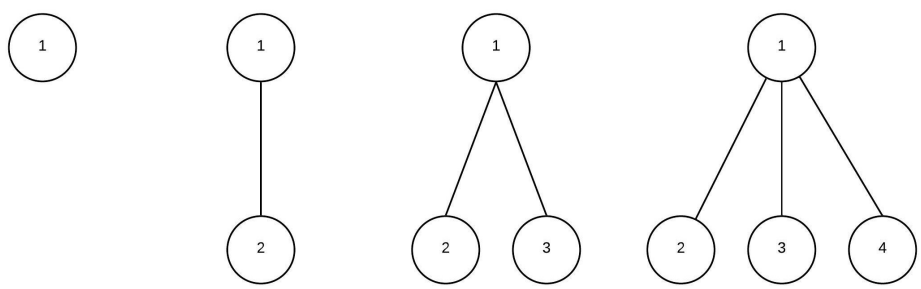
(i)

Let us consider a graph with n vertices. The possible degrees in such a graph are $0, 1, 2...n-1$. Now, we see that there are n possible choices of degrees and we may be tempted to think that we can assign one to every vertex. However, having a degree of $n-1$ implies that the vertex is connected to every other vertex, but we also have a vertex with degree 0, which means it is not connected to any other vertex. Since this is a clear contradiction, we conclude that the graph cannot contain two vertices with degrees 0 and $n-1$, it can have at most one of them. Therefore, we now have just $n-1$ degrees to assign to n vertices and by pigeonhole principle, at least two of them now much have the same degree.

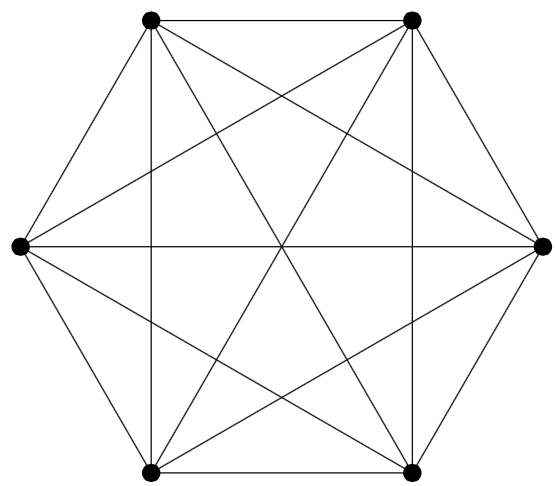
(ii)
Let us look at a table which compares the number of vertices and the required sum of degrees

Vertices	$\sum_i^n d_i = 2n-2$
1	0
2	2
3	4
4	6
5	8
...	...

It is clear that for every additional vertex, the sum of degrees increases by **2**. So the tree for two vertices can be constructed by simply joining two vertices, and for every subsequently added vertex, we can simply connect it to an assigned 'root' node. The figure given below illustrates the same for n = 1, 2, 3 and 4. Note that I am counting both in-degrees and out-degrees as a degree of the vertex as they are not specified separately in the question.



(iii)
For reference, let us look at a complete graph of six vertices.

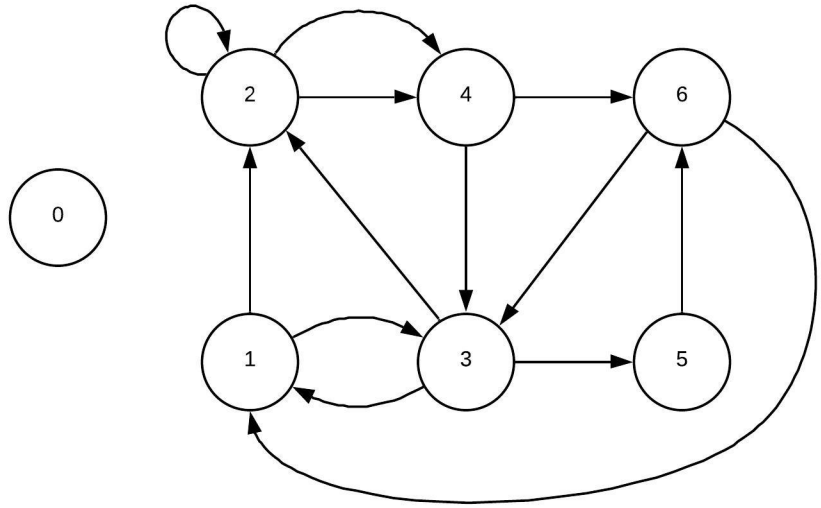


This is pretty trivial to show. Since all vertices are connected to each other, as soon as there are three vertices of the same colour, we can say that a triangle can easily be constructed of that colour. So if we are using two colours, any graph with at least five vertices will have to have at least three vertices of one colour, and hence a triangle can be made. Up to four vertices, this situation may be avoided by colouring two vertices red and two blue. But as soon as the fifth vertex comes into the picture, it needs to be given a colour, and will cause a blue/red triangle to be formed. Hence, a complete graph with **six** vertices will definitely have a red/blue triangle.

Question 4

A directed graph is Eulerian if the in degree equals out degree for every vertex. Show that an Eulerian graph admits a tour where every edge is visited exactly once. Design an efficient (linear time) algorithm to find such a tour.

To explain my algorithm for this question, I will be using the following graph for reference.



Clearly, the in-degree is equal to the out-degree for every vertex, although they are not the same for every vertex. According to the algorithm I propose, we make a table with the out-degrees of every vertex.

Vertex	Out-degree
0	0
1	2
2	2
3	3
4	2
5	1
6	2

Then we select any vertex with a non-zero out-degree (any vertex except 0 in our case) and start a DFS from it. Let us say we choose vertex 1. So basically, we choose any outgoing edge and mark it as visited, and reduce the outgoing edges, for this vertex in the table we made. We now end up reaching a new vertex. The table now looks like this.

Vertex	Out-degree
0	0
1	1
2	2
3	3
4	2
5	1
6	2

If this new vertex has at least one outgoing edge according to the table, we take any one of them and repeat this process until we get to a node where there is no outgoing edge left in

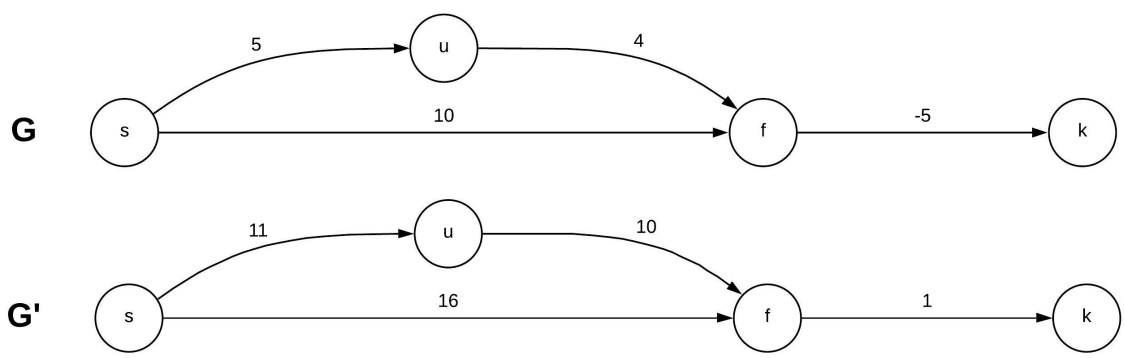
the table. At this point, we add this vertex to the front of a list we maintain for keeping the order of the 'tour', and we backtrack to the last visited vertex, where we again check if there is any outgoing edge left or else, we backtrack. This process is repeated until we have backtracked all the way until the starting vertex and all the out-degrees have become zero according to our table. The content of the list at this point show the order in which the vertices must be visited to produce on the the valid tours for this graph. Since the DFS at each vertex is a linear time algorithm, the overall algorithm also runs in linear time.

Question 5

Given a graph G with negative weights (no negative cycles), we want to transform it to another equivalent graph G^1 that preserves the shortest paths of G but doesn't contain any negative weights.
(i) If we add to all edges a weight greater than the largest negative weight, will shortest paths be preserved?
(ii) Let $d(v)$ be equal to the shortest path distance to v from source vertex s. Suppose we add to every edge (u, v), the weight $d(u) - d(v)$, i.e., the new weight $w^1(u, v) = w(u, v) + d(u) - d(v)$.
Then show that

1. $w^1(u, v) \geq 0$
2. **Between all pairs of vertices x, y, for two distinct paths P_1 and P_2, $w(P_1) \geq w(P_2)$ iff $w^1(P_1) \geq w^1(P_2)$.**

i
Let us consider the following example



Let graph G represent a graph with negative weights but no negative cycles. Let us look at the shortest path from vertex s to vertex f. We see that the path $s->u->f$ is the shortest since its 'cost' is 9 units. Another path $s->f$ exists but we see that it is more expensive.
Now we add a weight greater than the largest negative weight to all the edges. So, in this case, let us add a weight of 6 to all the edges to get the graph represented by G^1. We see that

while the relative ordering of weights is preserved, the path with more intermediate steps is penalised disproportionately. Specifically, in this example, the path $s->u->f$ faces a penalty of $6*2=12units$ while the path $s->f$ is penalised by just $6units$, since it has lesser connecting edges. This results in the shortest path of G no longer being the shortest path of G^1, and hence this technique to eliminate negative weights is clearly flawed.

ii

1. Since $d(u)$ and $d(v)$ are the shortest distances to u and v respectively, from the source vertex s, it is clear that a path from u to v has a lower bound of $d(v)-d(u)$. Had this not been the case then $d(v)$ would not be the shortest path from u to v.

   As per the question, we have to add $d(u)-d(v)$ to every edge (u,v). This can be seen as an operation equivalent to subtracting $d(v)-d(u)$, which we have established to be the shortest possible path between u and v.

   Hence, subtracting this weight from the weight on the edge (u,v) would always result in a non-negative weight and hence it can easily be said that $w^1(u,v)\geq 0$

2. Let us consider an arbitrary pair of vertices (x,y). We are given that $w(P_1)\geq w(P_2)$. Adding the constant $d(x)-d(y)$ to both sides of the inequality, we can easily generate the relationship $w^1(P_1)\geq w^1(P_2)$. Similarly, the other side can be proved by subtracting $d(x)-d(y)$ from both sides of the inequality $w^1(P_1)\geq w^1(P_2)$.